OPENCV • PYTHON

Corrigé enseignant – Initiation à OpenCV

Document enseignant | Corrigés, conseils pédagogiques et barème

Déroulement suggéré

• Durée indicative : 2 h à 2 h 30. Prévoir 10 minutes d'installation, 20 minutes pour les TP 1 et 2, 20 minutes pour les TP 3 et 4, 20 minutes pour le TP 5, puis 30 à 40 minutes pour le mini-défi.

• Faire manipuler les étudiants après chaque démonstration. Demander systématiquement de vérifier que l'image a bien été chargée avant de la traiter.

• Le fichier haarcascade_frontalface_default.xml doit être placé dans le même dossier que le script, ou son chemin doit être indiqué explicitement.

Corrigé — Questions et notions attendues

• TP 1 : imread() lit l'image depuis le disque et retourne une matrice de pixels. waitKey(0) attend indéfiniment une touche. Si le chemin est incorrect, imread() retourne None.

• TP 2 : l'image en niveaux de gris ne contient plus les informations de couleur ; elle représente principalement l'intensité lumineuse.

• TP 3 : dans resize(), le tuple est donné dans l'ordre (largeur, hauteur).

• TP 4 : le flou atténue les détails et le bruit en moyennant les pixels voisins.

• TP 5 : les lignes blanches indiquent les zones où l'algorithme détecte des changements importants d'intensité, souvent les frontières des objets.

• TP 6 : le classificateur Haar recherche des motifs visuels correspondant à des visages. La détection peut manquer des visages ou produire des faux positifs selon la qualité, l'orientation et l'éclairage.

Code de correction — TP 1 à TP 6

• Les scripts ci-dessous constituent une solution possible. Ils vérifient le chargement des images et le chargement du classificateur avant de poursuivre.

Barème indicatif sur 20

• Installation et organisation du projet : 2 points.

• Lecture et affichage d'image : 3 points.

• Conversion en niveaux de gris : 2 points.

• Redimensionnement et flou : 4 points.

• Détection des contours : 3 points.

• Détection de visages : 4 points.

• Qualité du code, commentaires et explication : 2 points.

Erreurs fréquentes à expliquer

• Image introuvable : vérifier le nom, l'extension et le répertoire de travail.

• Fenêtre qui se ferme immédiatement : ajouter waitKey(0) avant destroyAllWindows().

• Fichier XML introuvable : vérifier le chemin du fichier Haar Cascade et detecteur.empty().

• Couleurs inattendues : OpenCV lit généralement les images en BGR, et non en RGB.

• Dimensions inversées : cv2.resize() attend (largeur, hauteur), contrairement à l'ordre habituel hauteur/largeur des tableaux.

TP 1 — Lire et afficher

```
import cv2

image = cv2.imread("image.jpg")

if image is None:
    print("Erreur : image.jpg introuvable")
else:
    cv2.imshow("Mon image", image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 2 — Niveaux de gris

```
import cv2

image = cv2.imread("image.jpg")
if image is None:
    print("Image introuvable")
else:
    gris = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    cv2.imshow("Originale", image)
    cv2.imshow("Niveaux de gris", gris)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 3 — Redimensionner

```
import cv2

image = cv2.imread("image.jpg")
if image is not None:
    petite = cv2.resize(image, (400, 300))
    cv2.imshow("Originale", image)
    cv2.imshow("Redimensionnee", petite)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 4 — Flou

```
import cv2

image = cv2.imread("image.jpg")
if image is not None:
    floue = cv2.GaussianBlur(image, (7, 7), 0)
    cv2.imshow("Originale", image)
    cv2.imshow("Image floue", floue)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 5 — Contours

```
import cv2

image = cv2.imread("image.jpg")
if image is not None:
    gris = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    contours = cv2.Canny(gris, 100, 200)
    cv2.imshow("Contours", contours)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

TP 6 — Détection de visages

```
import cv2

detecteur = cv2.CascadeClassifier(
    "haarcascade_frontalface_default.xml"
)

image = cv2.imread("personnes.jpg")
if image is None:
    print("Image introuvable")
elif detecteur.empty():
    print("Fichier XML du détecteur introuvable")
else:
    gris = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    visages = detecteur.detectMultiScale(
        gris, scaleFactor=1.1, minNeighbors=5
    )
```

```
for (x, y, w, h) in visages:
    cv2.rectangle(image, (x, y), (x+w, y+h), (0, 255, 0), 2)

print("Nombre de visages :", len(visages))
cv2.imshow("Detection de visages", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Ressource technique

Documentation officielle OpenCV : https://docs.opencv.org/4.x/

Le fichier Haar Cascade est disponible dans le dépôt officiel OpenCV, dossier data/haarcascades.